

CO2 AT3
ASSIGNMENT-3

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Name: M. Pranati
Reg No:192312298
Course Code: Slot B-IT0614
Course Name: Machine Learning

Faculty Name

JASPHIN JENI SHARMILA P

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Introduction

A decision tree is a supervised machine learning algorithm used for classification and regression tasks. It builds a tree-like structure by splitting the dataset based on feature values. Sometimes, a decision tree achieves very high accuracy on the training dataset but performs poorly on unseen (test) data. This indicates that the model has failed to generalize well and is suffering from overfitting or other issues.

Possible Causes

1. Overfitting

Overfitting occurs when the decision tree learns not only the important patterns in the training data but also the noise and random variations. As a result, it performs extremely well on training data but poorly on new data.

Effects:

- High training accuracy.
- Low testing accuracy.
- Poor generalization.

2. Deep Tree Structure

A decision tree with many levels becomes very complex. It creates numerous branches to classify every training example perfectly, resulting in a highly specific model that cannot handle unseen data effectively.

Effects:

- Increased model complexity.
- High variance.
- Reduced prediction accuracy on new samples.

3. Noisy Data

Training data may contain incorrect labels, missing values, or irrelevant information. The decision tree may attempt to fit these noisy records, leading to inaccurate predictions.

Examples of noise:

- Incorrect class labels.
- Missing attribute values.

- Outliers.
- Duplicate records.

4. Irrelevant Features

Some features may not contribute to prediction but still influence tree construction, making the model unnecessarily complex.

5. Insufficient Training Data

A small dataset may not represent the actual problem adequately, causing the model to memorize the available samples instead of learning general patterns.

Debugging Steps

1. Analyze Training and Testing Accuracy

Compare training accuracy with testing accuracy.

- High training accuracy and low testing accuracy indicate overfitting.
- Similar accuracies indicate good generalization.

2. Visualize the Decision Tree

Inspect the tree structure.

If the tree contains too many branches and nodes, it is likely overfitting

3. Check Data Quality

Verify the dataset for:

- Missing values.
- Duplicate records.
- Outliers.
- Incorrect class labels.

Clean the data before training.

4. Analyze Feature Importance

Identify which features contribute most to classification.

Remove irrelevant or redundant attributes.

5. Evaluate Confusion Matrix

The confusion matrix helps identify which classes are being misclassified and whether prediction errors occur for specific categories.

Optimization Methods

1. Pruning

Pruning removes unnecessary branches from the decision tree.

Types of Pruning:

- **Pre-pruning:** Stops tree growth early by limiting depth or minimum samples required for a split.
- **Post-pruning:** Builds the complete tree first and then removes branches that do not improve performance.

Benefits:

- Reduces overfitting.
- Improves testing accuracy.
- Produces a simpler and more interpretable model.

2. Cross-Validation

Cross-validation divides the dataset into multiple folds. The model is trained and tested on different folds, and the average performance is computed.

Advantages:

- Provides a reliable estimate of model performance.
- Reduces bias due to a single train-test split.
- Improves model robustness.

3. Feature Selection

Feature selection removes unnecessary attributes and retains only the important features.

Common techniques include:

- Correlation Analysis.
- Information Gain.
- Chi-Square Test.
- Recursive Feature Elimination (RFE).
- Principal Component Analysis (PCA).

Benefits:

- Reduces model complexity.
- Improves accuracy.
- Decreases training time.

4. Limit Tree Depth

Restrict the maximum depth of the decision tree to prevent excessive branching and reduce overfitting.

5. Set Minimum Samples per Leaf

Increase the minimum number of samples required in a leaf node so that the tree does not create branches based on very few examples.

6. Ensemble Methods

Algorithms such as **Random Forest** and **Gradient Boosting** combine multiple decision trees to improve prediction accuracy and reduce overfitting.

Benefits After Optimization

- Better generalization to unseen data.
- Reduced overfitting.
- Improved prediction accuracy.
- Simpler and more interpretable decision tree.
- More reliable performance in real-world applications.

Conclusion

Poor performance of a decision tree on unseen data is mainly caused by overfitting, deep tree structures, noisy data, and irrelevant features. By applying debugging techniques such as analysing the tree structure, checking data quality, and evaluating feature importance, the root cause can be identified. Optimization methods such as pruning, cross-validation, feature selection, limiting tree depth, and ensemble learning significantly improve the model's ability to generalize and produce accurate predictions on new data.

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Introduction

A Convolutional Neural Network (CNN) is a deep learning model widely used for image classification tasks. CNNs automatically learn hierarchical image features using convolutional and pooling layers. However, during training, the model may produce unstable accuracy and fluctuating loss values across epochs. Such instability affects the convergence and overall performance of the network.

Possible Causes

1. Improper Data Augmentation

Data augmentation increases the diversity of training images through transformations such as rotation, flipping, zooming, and cropping.

If augmentation is excessive or unrealistic:

- Images become distorted.
- Important features may be lost.
- Training becomes unstable.

2. Vanishing Gradients

In deep neural networks, gradients become extremely small while propagating backward through many layers.

Effects:

- Very slow learning.
- Weights are not updated effectively.
- Poor convergence.
- Low classification accuracy.

3. Batch Normalization Errors

Batch normalization stabilizes the distribution of inputs in each layer.

Incorrect placement or implementation may cause:

- Oscillating training accuracy.
- Slow convergence.
- Unstable loss values.

4. High Learning Rate

A learning rate that is too high causes large weight updates, leading to overshooting of the optimal solution and fluctuating training performance.

5. Small Batch Size

Very small batch sizes produce noisy gradient estimates, resulting in unstable training.

6. Insufficient Training Data

A limited dataset may not represent all image variations, causing poor generalization and inconsistent performance.

7. Poor Weight Initialization

Improper initialization can cause gradients to vanish or explode, making training unstable.

Debugging Strategies

1. Monitor Training Curves

Plot training and validation accuracy and loss for each epoch.

Observe whether:

- Loss decreases consistently.
- Accuracy improves steadily.
- Validation performance follows training performance.

2. Verify Data Augmentation

Check whether transformations preserve meaningful image features.

Avoid excessive:

- Rotation.
- Cropping.
- Zooming.
- Brightness adjustments.

3. Inspect Batch Normalization Layers

Ensure that batch normalization layers are correctly placed after convolutional layers and before activation functions.

4. Check Gradient Values

Monitor gradients during training to detect vanishing or exploding gradients.

5. Validate Dataset

Inspect the dataset for:

- Incorrect labels.
- Duplicate images.
- Missing images.
- Class imbalance.

6. Monitor Learning Rate

If loss fluctuates significantly, reduce the learning rate or use a learning rate scheduler.

Optimization Techniques

1. Batch Normalization

Normalize intermediate feature maps to stabilize learning and accelerate convergence.

2. Proper Data Augmentation

Use realistic augmentation methods:

- Horizontal flipping.
- Small rotations.
- Zooming.
- Translation.
- Brightness adjustment.

This increases dataset diversity while preserving image characteristics.

3. ReLU Activation Function

Use the **ReLU** activation function in hidden layers to reduce the vanishing gradient problem and improve training speed.

4. Dropout Regularization

Randomly deactivate neurons during training to prevent overfitting and improve model generalization.

5. Learning Rate Scheduling

Gradually reduce the learning rate during training using scheduling techniques to stabilize convergence.

6. Early Stopping

Stop training when the validation loss no longer improves to prevent overfitting and unnecessary computation.

7. Transfer Learning

Use pre-trained CNN models such as:

- VGG16
- ResNet50
- MobileNet
- InceptionV3

These models improve accuracy, especially for smaller datasets.

8. Increase Dataset Size

Collect more training images or apply balanced augmentation to improve model robustness and reduce overfitting.

Benefits After Optimization

- Stable training across epochs.
- Faster convergence.
- Higher classification accuracy.

- Reduced overfitting.
- Better generalization to unseen images.
- Improved robustness and reliability.

Conclusion

Unstable CNN training is commonly caused by improper data augmentation, vanishing gradients, batch normalization issues, unsuitable learning rates, and limited datasets. By carefully monitoring training behaviour, validating the dataset, and applying optimization techniques such as batch normalization, ReLU activation, dropout, learning rate scheduling, early stopping, and transfer learning, the training process becomes more stable and the model achieves better convergence and higher image classification accuracy.